ClusterMesh Service v2

A new format for ClusterMesh services!

Limitation of the existing ClusterService

Existing ClusterService v1 format

```
{
  "name": "echo",
  "namespace": "test",
  "cluster": "cluster2",
  "clusterID": 2,
  "hostnames": {
    "20.0.0.2": "echo-1"
  },
  "zones": {
    "20.0.0.2": {
      "zone": "eu-west-1",
      "forZones": [{"name": "eu-west-1"}]
    }
  },
  "backends": {
    "20.0.0.2": {
      "tcp": {"Protocol": "TCP", "Port": 9090}
    }
  },
  # everything below this is unused
  "includeExternal": true, "shared": true, "frontends": {},
  "labels": {},"selector": {}
}
```

- Verbose
  - Format duplicate info per IP and ports in the “backends” fields
  - JSON encoding
- No conditions info
  - Endpoints with ready=false & serving=false are excluded at export time
  - This means no graceful termination :(

Max size from etcd

A value in etcd is **limited to 1.5MiB** -> with a service with 2 ports that's a max of **~10 000 backends** (per service per cluster)

Each fields we want to add will decrease that max number

For instance adding conditions and EndpointSlice name could reduce that maximum to **~8 000 backends** (per service per cluster)

All the backends from all clusters/services are in a **eBPF map limited to 64k by default** + Kubernetes is designed to **have up to 5k nodes upstream** (you could have dual stack services though!) -> 8k backends per cluster per service is **still relatively ok**

Bandwidth on ClusterService updates

Napkin math - a ClusterService with 5 000 backends and 2 ports each is **~786KiB**:

- **If each update** from this ClusterService need to propagated to 10 000 cilium-agents -> **~7.5GiB total** / A Deployment rollout could easily be ~100+ GiB total

We are in a ~ similar situation as **Kubernetes** when they switched **from Endpoints to EndpointSlices** except that we could have more nodes in a mesh than a single cluster!

Insertion performance on churn in statedb

There is a different code path from clustermesh-apiserver vs from kubernetes-apiserver

Backends	clustermesh (µs)	clustermesh w/o JSON decoding (µs)	loadbalancer k8s (µs)	Ratio (clustermesh w/o JSON/k8s)
1	44	35	4	9x slower
100	629	271	4	68x slower
1 000	6 349	2 626	7	375x slower
5 000	36 861	15 831	30	528x slower
10 000	78 810	44 289	70	633x slower

Also note that there is no ingestion coalescing happening on Cluster Mesh

How to fix this?

ClusterService v2 proposal

```go
type ClusterServiceV2 struct {
        Cluster string `json:"cluster"`
        ClusterID uint32 `json:"clusterID"`
        Namespace string `json:"namespace"`
        Name string `json:"name"`
        // Note that not every field from the EndpointSlice will
        // be populated (for instance, fields from TypeMeta and most
        // fields from ObjectMeta)
        EndpointSlices []*slim_discovery_v1.EndpointSlice `json:"endpointslices"`
}
```

- Embed EndpointSlice for simplicity and better format / less data duplication

- Shared ingestion code path with k8s loadbalancer reflector -> better ingestion perf + easier evolution and maintenance

Compression with zstd

Compress ClusterServiceV2 -> fix bandwidth and any etcd max size constraints

zstd on large ClusterService v1(~5 000 backends) struct **compress by ~50x** with just **~5%** more CPU usage compared just decoding the JSON

Still not entirely ideal perf wise on decoding perf vs a chunked/slice approach

BUT **decoding is parallelized per cluster** and Go's json/v2 might (?) help + it's a **simpler** than introducing chunking/slices with what we have today

Caveat

Including endpoints conditions -> **more churn**

Limited at 20qps by default to clustermesh etcd for all resources

If we go over that limit at export time it will coalesce updates. But it still means we are constrained by a “quota” overall

We could **feature flag conditions updates**
-> start with the same qps and then study how we could enable it by default

Open to other ideas!

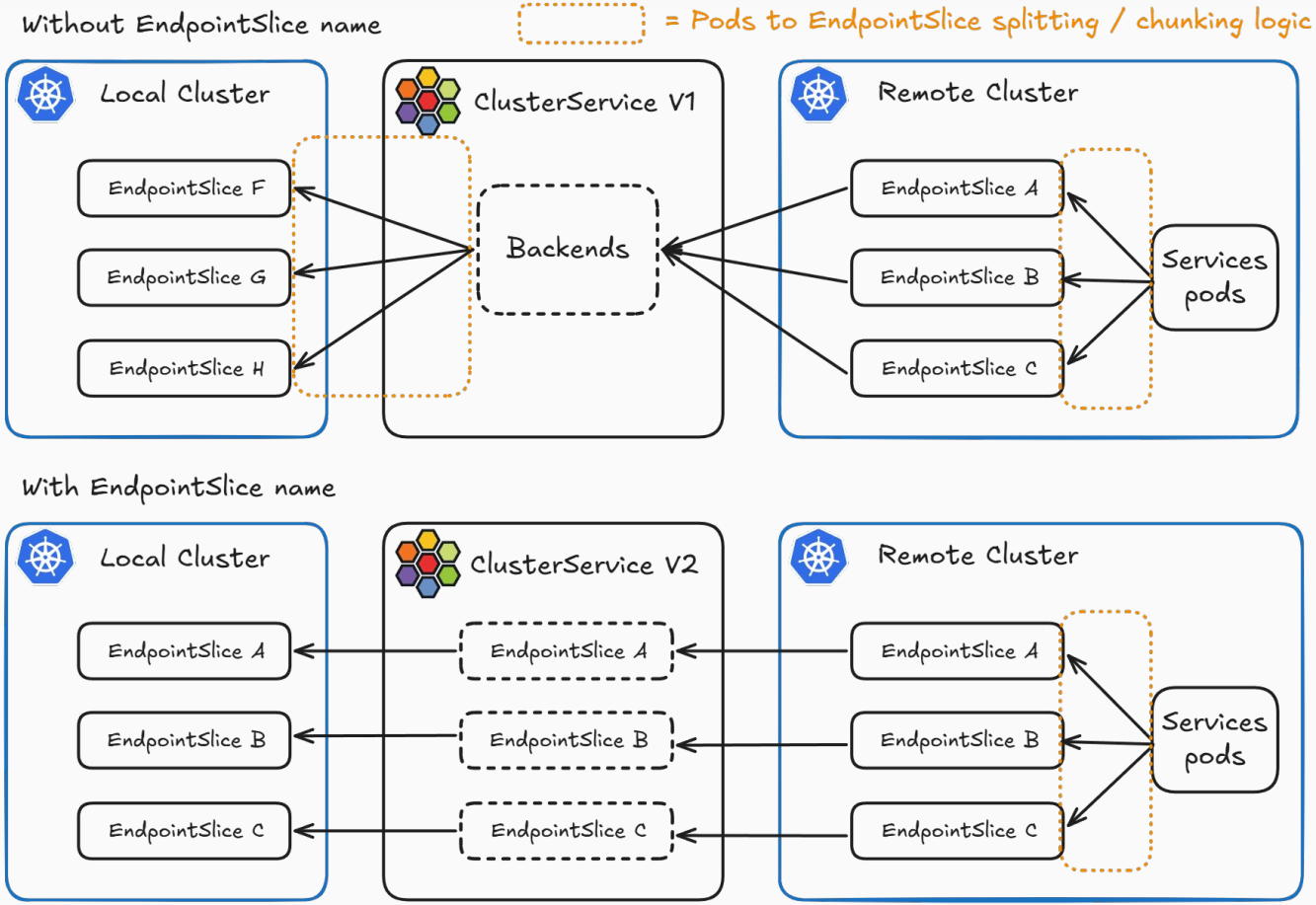
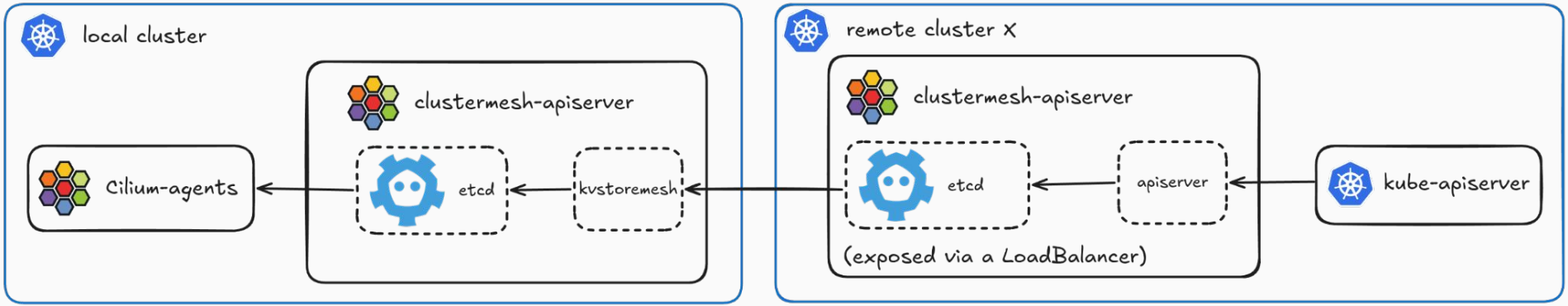
Initially started to design with ProtoBuf but this would make it more complex to inspect the etcd values (and more complex hive test scripts)

We are technically less constrained than k8s on encoding

Not planning to change encoding of other structs (most other structs are also used by kvstore + they are constrained in size)

Bonus

EndpointSliceSync simplification



Cluster Mesh architecture